

07-26 Lecture: Client-Driven Engineering Program and AI Tooling

Date Time: 2026-07-26 17:11:07

Location: [Insert Location]

Instructor: Carrie

Summary

This series of lectures and hands-on workshops introduces a client-driven engineering program for the “University of College” freshman class. The program’s goal is to teach students how to build and deploy small software tools (apps, websites, utilities) for real-world clients. Over the course of the sessions, the instructor, Carrie, guides students through setting up a complete development environment, covering foundational concepts from command-line usage to version control with Git and GitHub, and integrating AI tools like the Claude CLI and the Cursor IDE.

The first session outlines the program structure, client assignments (for clients like Amy, Jenny, Ismailia, and Nancy), and an AI-enabled workflow designed to streamline development from client communication to deployment on platforms like Vercel. Students begin by learning terminal basics (`ls` , `cd` , `mkdir` , `pwd`), creating a GitHub account, and cloning the class repository (“U of C”).

The second session focuses on core software development concepts, explaining the role of an IDE (using Cursor as an example) and the distinction between local storage and cloud computing. The instructor uses Apple Photos as a one-to-one sync analogy before detailing GitHub’s more complex “one-to-many” model for collaborative coding. The four essential Git commands— `git pull` , `git push` , `git add` , and `git commit` —are introduced, and students practice a full workflow: creating a file, adding it to staging, committing it locally, and preparing to push it to the remote repository.

The final session is a practical workshop where students apply their Git knowledge by pulling and pushing code, troubleshooting common issues like divergent branches and permissions. The main focus is on installing the Claude AI command-line interface, which involves using `curl` , configuring system paths, and launching the tool from the terminal. The instructor emphasizes that mastering these underlying tools—Git, GitHub, and a locally running AI—is critical infrastructure for

all future development. The sessions conclude with assignments for students to use the Claude CLI to self-teach terminal and Git fundamentals, preparing them for the next phase: onboarding to actual client projects.

Knowledge Points

1. Program Overview and Goals

- **Client-Driven Engineering Model:** Students act as engineers for clients such as Ismailia (PRI), Jenny (bookkeeping), Amy (real estate), and Nancy (nurse practitioner). The goal is to build small tools to solve their operational problems and learn to take prototypes to production-ready software with ongoing support.
- **Scaling and Business Building:** The program teaches how to scale solutions by aggregating common problems (e.g., for multiple real estate agents) and reusing code. A referral chain (e.g., Amy refers Richard, who refers Bob) demonstrates how solving one client's problem can build a small business.

2. AI-Enabled Workflow and Tooling

- **Communication-to-Code Pipeline:** An AI tool is being developed to transcribe client communications (calls, texts, emails), generate project scopes, and automatically update the GitHub code repository. Students will be responsible for validating the AI's actions, including database updates and deployments on platforms like Vercel.
- **Development Environment:** The core stack includes the command line, GitHub for version control, the Cursor IDE (an AI-enhanced version of VS Code) for writing code, and the Claude CLI for interactive learning and task execution directly within the terminal.

3. Foundational Technical Concepts

- **Terminal and CLI Basics:** The terminal provides text-based access to the operating system, essential for developer and AI tooling. Key commands covered include `ls` (list contents), `cd` (change directory), `mkdir` (make directory), and `pwd` (present working directory). A strong recommendation was made to avoid spaces in file and folder names.
- **Cloud vs. Local Storage:** The "cloud" is defined as remote data storage (e.g., on Apple's or GitHub's servers), contrasting with "local" storage on a personal device. While simple cloud services like Apple Photos offer one-to-one sync, GitHub provides a many-to-one model, allowing multiple developers to collaborate on a single codebase.

4. Git & GitHub Version Control

- **Core Purpose:** Git is a command-line tool that manages the flow of code between a local machine and a remote GitHub repository. It maintains a version history and resolves complexities arising from multiple contributors working simultaneously.
- **Key Commands:**
 - `git clone` : Used once at the beginning to copy a remote repository to a local machine.
 - `git pull` : Downloads the latest changes from the remote repository to the local machine.
 - `git status` : Shows the current state of the repository, including untracked, modified, and staged files.
 - `git add <filename>` : Adds specified files to the “staging area,” preparing them for a commit.
 - `git commit -m "message"` : Saves staged changes to the local repository’s history with a descriptive message.
 - `git push` : Uploads locally committed changes to the remote repository on GitHub, making them visible to others.
- **Troubleshooting:** Common issues were addressed, such as the “divergent branches” error (fixed with `git config pull.rebase=false` for a first-time pull) and permissions errors that prevent read/write access.

5. Claude CLI Installation and Usage

- **Installation:** Students were guided to download the Claude CLI from `claude.ai/downloads` using a `curl` command in the terminal. Post-install steps included fixing path issues by following instructions printed in the terminal output.
- **Interactive Learning:** Once installed and launched (`claude`), the CLI can be used as a self-teaching tool directly in the terminal. The instructor assigned students to prompt Claude to “teach me terminal basics” and generate cheat sheets for Git and terminal commands.
- **Strategic Importance:** The instructor stressed that running AI tools like Claude locally via the terminal, combined with Git fluency, is a critical skill that distinguishes a proficient developer and forms the foundation for all subsequent AI-related work in the course.

6. Initial Project Assignments

- **Student-Client Pairings:**

- **Jun:** Amy (real estate) and Richard (real estate referral).
- **Lydia:** Jenny (bookkeeping).
- **JL:** Ismailia (PRI) and the boot camp tool.
- **AJ:** Nancy (nurse practitioner, credentialing).

- **Next Steps:** The instructor will provide existing prototypes for students to begin their work. The next session will focus on onboarding to the actual client builds.

Questions

- [Insert Question/Confusion]

Assignments

- [] 1. Create a GitHub account and install the Cursor IDE.
- [] 2. Practice terminal commands (`ls` , `cd` , `mkdir` , `pwd`) to navigate directories and create folders.
- [] 3. Clone the class “U of C” repository from the instructor’s GitHub account (“Carrie Keller”) using `git clone` .
- [] 4. Inside the cloned repository, create a new personal file (e.g., `yourname.txt`).
- [] 5. Practice the full Git workflow: use `git add` to stage the new file, `git commit -m "message"` to save it locally, and prepare to use `git push` to upload it.
- [] 6. Practice pulling changes from the remote repository using `git pull` .
- [] 7. Install the Claude CLI using the `curl` command from `claude.ai/downloads` and configure it to run in your terminal.
- [] 8. Use the Claude CLI to self-teach and reinforce concepts. Prompt it with: *“Teach me terminal basics and setting up a repository on GitHub.”*
- [] 9. Ask Claude to generate a **cheat sheet** and **worksheets** covering terminal and Git commands for future reference.
- [] 10. Await and review the client project prototypes provided by the instructor to prepare for the onboarding session.